DLSCMS

Defence Logistics and Supply Chain Management System

Object Oriented Programming mini project: a Java model that moves stock between bases, and a study of how well dispatch rules work

Vedant Pathak (R045) Dhairya Shah (R057) Kanishak Thakran (R046)
Faculty: Dr. Sakshi Ranjan | SVKM's NMIMS MPSTME, Mumbai

The problem: right stock, right place, in time

The situation

Bases hold stock of fuel, food, medical, ammunition and spares.

Requests arrive with a priority from ROUTINE to CRITICAL.

A donor base and a vehicle must be chosen every time.

What we built

A Java system with 16 terminal commands: bases, inventory, requests, fleet, deliveries, reports.

A simulated clock that moves hour by hour.

Save and load to CSV files.

What we asked

Does the choice of donor base matter?

Does the choice of vehicle matter?

How does coordination compare with a random baseline, and does it scale?

Architecture: nine packages, one facade

ui	Bloomberg-style terminal: 16 function codes
service	LogisticsManager (Facade), Inventory, Request, Fleet, Delivery, Report
strategy	Donor and vehicle selection rules, swappable at run time
model + exception	Base, Supply, Request, Delivery, Vehicle hierarchy; checked exceptions
persistence + util + sim	CSV repositories, Registry<T>, SimClock, experiment runner

Where each OOP idea shows up

1

Encapsulation

Private state, validated changes, enums for status

2

Inheritance

Abstract Vehicle: Truck, FuelTruck, MedicalVan, Helicopter

3

Polymorphism

Each vehicle estimates its own travel time and what it can carry

4

Abstraction

Interfaces Trackable, Deliverable, Persistable

5

Generics

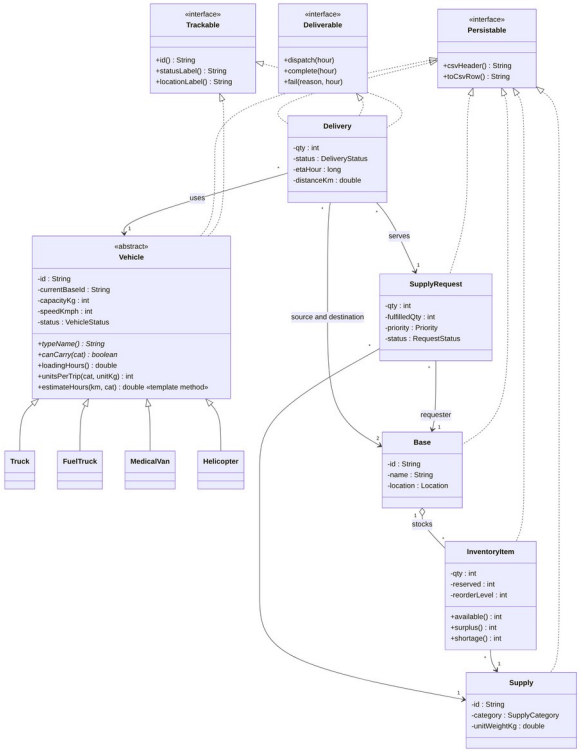
Registry<T> and CsvRepository<T extends Persistable>

6

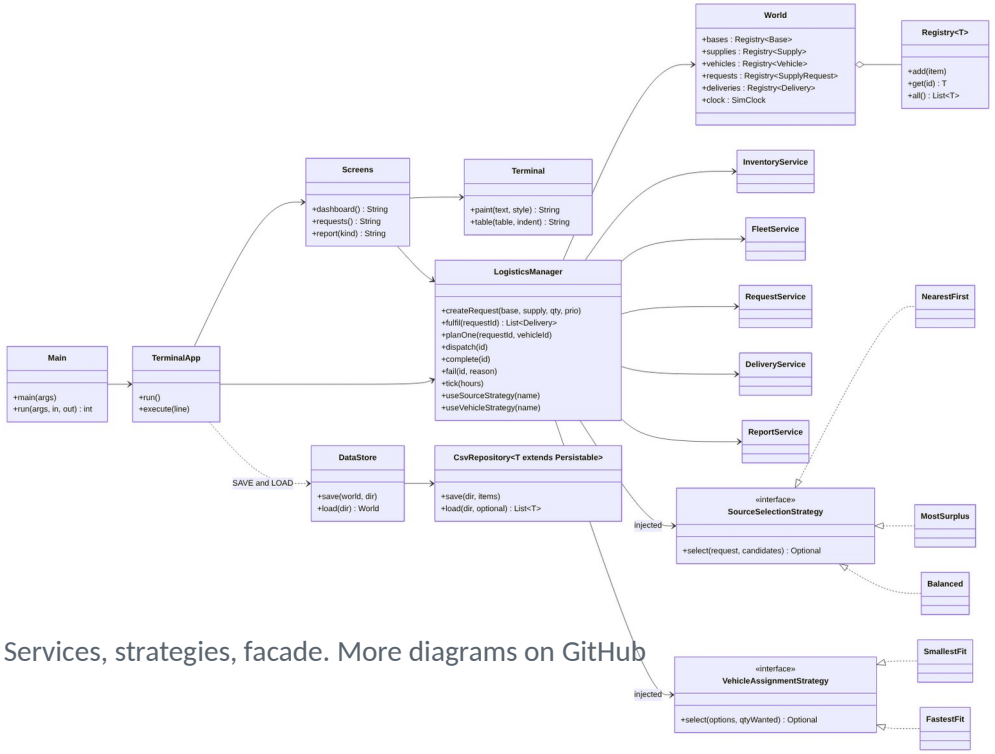
Exceptions

Checked LogisticsException family, no silent failures

UML class diagrams



Domain model



Services, strategies, facade. More diagrams on GitHub

Design choices that make the rules swappable

Strategy

NEAREST, SURPLUS, BALANCED donors; SMALLEST, FASTEST vehicles; RANDOM as a baseline. Change with STRAT.

Facade

LogisticsManager is the single entry point for the UI, the scripts and the simulator.

Template Method

Vehicle.estimateHours is fixed; each subclass supplies speed and what it may carry.

Domain safety rules

Donors keep their reorder level. Stock is reserved on plan, removed on dispatch, added on delivery, returned on failure.

Live demo

Scenario: Pune needs 200 medical kits, urgently

1. `run.bat` (opens the example network; `--demo` auto-plays it)
2. `REQ NEW PUN MED-KIT 200 URGENT` then `REQ APPROVE`
3. `REQ FULFIL` (donor and vehicle chosen by the strategies)
4. `DLV DISPATCH` then `TICK 12` (time moves only on TICK)
5. `RPT KPI` then `STRAT SRC BALANCED` (swap the rule)

How we tested the dispatch rules

1,080
simulation runs

3 x 30
sizes x datasets

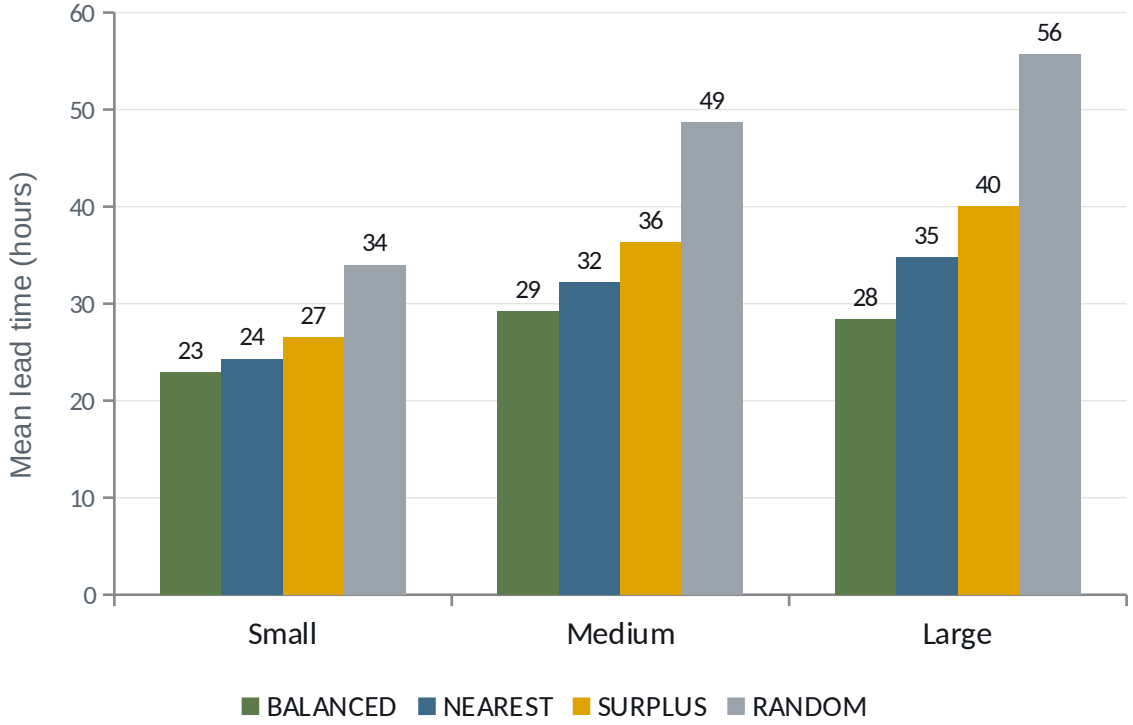
12
strategy pairs

74 / 93
tests significant after correction

Metrics: fulfilment rate, lead time (request to full delivery), critical requests within 24 h, distance, vehicle use.

Honest limits: synthetic data, straight-line distances, constant speeds, no breakdowns.

Result 1: BALANCED is fastest from medium size upward



6.3 h

faster than NEAREST on large ($p < 0.001$ after correction)

NEAREST drives the fewest kilometres; SURPLUS about 42% more on large.

Result 2: coordination beats the random baseline

24 h vs 46 h

lead time, small

35 h vs 85 h

lead time, large

84% vs 32%

critical requests within 24 h, large

-42%

distance travelled, large

Coordinated = NEAREST donor + FASTEST vehicle. Baseline = random donor + random vehicle. The baseline is deliberately weak.

Quality: tests, reproducibility and honest limits

119 tests

21 test classes cover all 66 types. Run with one command (com.dlscms.RunAll).

Reproducible

Fixed seeds; a script re-runs the grid and compares every number with the saved results.

Known gaps

1-3% of requests stay open: donors keep a reserve and fuel trucks are scarce. We explain it, we did not remove it.

Summary and links

- Clean OOP design: facade, strategies, vehicle hierarchy, generics
- Swappable dispatch rules, measured over 1,080 runs
- BALANCED donor selection is fastest at scale; coordination clearly beats random
- Next: longer horizons, bigger fuel fleet, real-world data

Source code and UML on GitHub
https://github.com/withvedantpathak/defence_logistics_and_supply_chain_management_system_using_objectorientedprogramming

Thank you. Questions?